

Structured ASIC Placement Using Simulated Annealing

Farida Ragheb
Tony Gerges

May 19, 2026

Introduction

Given a netlist of heterogeneous cells and fixed pins, the placer assigns each cell to a legal site on a 52×52 grid while minimizing total Half-Perimeter Wire Length (HPWL). The core algorithm uses an SA schedule with incremental cost evaluation, type-legal move generation, and best-position tracking. Four optimization variants are evaluated: adaptive reheating, force-directed initialization, greedy lookahead move selection, and temperature-scaled range limiting. All variants are tested across five benchmark designs and five cooling rates (0.75, 0.80, 0.85, 0.90, 0.95).

1 Problem Formulation

1.1 Grid and Site Types

The chip grid is $N_y \times N_x = 52 \times 52$. Sites on rows 0, $N_y - 1$ and columns 0, $N_x - 1$ are reserved for fixed pins. All interior sites are assigned a type by the repeating 5×5 tile:

$$\text{type}(x, y) = M[(y - 1) \bmod 5][(x - 1) \bmod 5] \quad (1)$$

where M is the master tile matrix:

$$M = \begin{pmatrix} 0 & 1 & 0 & 2 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 2 & 3 & 0 & 2 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Tiling this pattern across the 50×50 core produces approximately 1500 T0 sites, 600 T1 sites, 300 T2 sites, and 100 T3 sites. T0 cells are the most common resource; T3 cells are the scarcest.

1.2 Cost Function

The cost metric is Half-Perimeter Wire Length (HPWL). For net k connecting cells with positions $\{(x_i, y_i)\}$:

$$\text{HPWL}_k = (\max_i x_i - \min_i x_i) + (\max_i y_i - \min_i y_i) \quad (2)$$

Total cost is $C = \sum_k \text{HPWL}_k$ summed over all nets.

1.3 Input Format and Parsing

Each netlist file is divided into three sections separated by blank lines after comment stripping. The parser processes the file in a single forward pass:

1. **Header.** A single line containing five integers: number of cells, number of nets, grid rows (N_y), grid columns (N_x), and number of fixed pins.
2. **Cell definitions.** One line per cell. A line ending in **P** defines a fixed I/O pin with explicit (x, y) coordinates and is stored with **fixed=True**. All other lines give a cell type tag of the form **Tk** (where $k \in \{0, 1, 2, 3\}$) and are stored with **fixed=False** and unassigned coordinates.
3. **Net definitions.** One line per net. The first integer gives the number of attached component IDs; the remaining integers on that line are the IDs themselves.

After parsing, the placer builds three secondary data structures that support efficient annealing:

- **sites_by_type:** a dictionary mapping each type k to the full list of (x, y) grid coordinates whose $\text{type}(x, y) = k$. Used once during initial placement.
- **empty_by_type:** a dictionary mapping each type k to a *set* of currently unoccupied sites of that type. Updated incrementally after every accepted move, giving $O(1)$ occupancy lookup.
- **cell_to_nets:** a dictionary mapping each component ID to the list of net indices that contain it. Constructed once in $O(N_{\text{pins}})$ time and used by the incremental delta evaluator to identify only the affected nets for each move candidate, avoiding a full HPWL recompute.

2 Algorithm

2.1 Initial Placement

Before annealing, each movable cell is assigned a random but type-legal starting position. Cells are first grouped by type. For each type, the corresponding site list from **sites_by_type** is shuffled and cells are assigned one-to-one. Fixed pins are never moved, so their coordinates were read directly from the netlist and they are placed into the grid during construction.

This strategy guarantees a fully legal starting placement with every cell on a compatible site, so the annealer never needs to perform a legality check during the run.

2.2 Simulated Annealing

The number of move attempts per temperature step is $20 \times N_{\text{cells}}$ for designs up to 500 cells, and scaled down for larger designs to keep runtime tractable.

At each temperature, a move is accepted if it reduces cost or with a probability. The best placement seen across the entire run is stored and restored at the end.

Early stopping applies if cost fails to improve for 20 consecutive temperature steps.

2.3 Incremental Cost Evaluation

Recomputing total HPWL from scratch after each move would be $O(N_{\text{nets}} \cdot N_{\text{cells}})$ per step. Instead, a `cell_to_nets` mapping lets the evaluator recompute only the nets affected by the cells involved in the move. The delta is computed by temporarily modifying coordinates and then restoring them, leaving no side effects.

2.4 Move Generation

A move is either a swap between two same-type cells or a relocation of a cell to an empty same-type site. Type legality is enforced by construction: cells only target cells or sites of the same type. Per-type cell lists are precomputed once before the annealing loop so move generation runs in $O(1)$.

2.5 2-opt Refinement

An optional post-annealing pass performs greedy local search. It iterates over all pairs of same-type cells and applies any swap that strictly reduces HPWL. This typically yields an additional improvement on top of the SA result.

3 Optimization Variants

Four variants of the base placer were developed and evaluated.

3.1 Adaptive Reheating

After each temperature step, the acceptance rate is computed as the fraction of attempted moves that were accepted. If the rate falls below 10%, the temperature is multiplied by a reheat factor, thereby capping it. Up to five reheats are permitted per run. The intent is to escape stagnation when the annealer freezes prematurely before finding a good solution.

3.2 Force-Directed Initialization

Instead of random initial placement, this variant uses a force-directed method. Each cell is given a continuous floating-point position; spring forces proportional to net centroid displacement are applied iteratively. The final continuous positions are discretized by sorting cells and sites by position and assigning them greedily.

3.3 Greedy Lookahead Move Selection

Rather than committing to the first move generated, this variant samples k candidate moves and selects the one with the lowest delta before applying the SA acceptance criterion. This biases the search toward improving directions at the cost of additional delta evaluations per step.

3.4 Temperature-Scaled Range Limiting

Move distances are restricted based on the current temperature. Early in the schedule moves can span the full grid; as temperature falls they are increasingly local. If no valid move is found within a range of attempts, the constraint is relaxed for that step.

4 Visualization

A separate visualization module renders placements using matplotlib. Each grid cell is color-coded by occupant type: green for T0, blue for T1, orange for T2, purple for T3, yellow for fixed pins, and dark for empty sites. Two modes are supported: a side-by-side before/after comparison of initial and final placements with HPWL annotations, and a progress snapshot view that captures the grid state at evenly spaced points through the annealing schedule.

5 Experimental Setup

5.1 Benchmark Designs

Five netlists of increasing size were used for evaluation, all targeting the same 52×52 grid with 20 fixed pins.

5.2 Cooling Rates Tested

Each placer variant was run at cooling rates $\alpha \in \{0.75, 0.80, 0.85, 0.90, 0.95\}$. Lower cooling rates reduce the temperature more aggressively and finish faster; higher rates allow finer exploration at the cost of longer runtime. Each configuration was run once with a fixed random seed.

6 Results

6.1 HPWL Improvement by Cooling Rate

Table 1 shows improvement percentages on `design_1_small` across all cooling rates. All variants improve with higher α , confirming that slower cooling gives the annealer more time to find better solutions.

Table 1: HPWL improvement (%) on `design_1_small` by variant and cooling rate

Variant	0.75	0.80	0.85	0.90	0.95
Baseline SA	66.4	66.6	67.1	67.5	68.0
Adaptive reheating	65.6	66.1	66.3	66.6	67.1
Force-directed init	49.2	49.6	49.3	52.4	53.3
Greedy lookahead	67.9	67.6	67.9	68.2	68.3
Range limiting	65.2	65.1	66.2	66.9	68.1

6.2 Mean Improvement Across Designs

Table 2 shows mean improvement percentages averaged over all five cooling rates for each variant and design. Improvement generally peaks on the medium and large designs then drops on dense and extreme, where the fixed grid becomes more congested and the search space harder to escape.

Table 2: Mean HPWL improvement (%) averaged over all cooling rates

Variant	Small	Medium	Large	Dense	Extreme
Baseline SA	67.1	73.4	73.2	64.2	57.6
Adaptive reheating	66.3	70.7	68.9	65.3	59.5
Force-directed init	50.8	54.5	53.2	50.8	49.3
Greedy lookahead	68.0	74.1	73.1	69.0	64.0
Range limiting	66.3	70.5	68.3	64.2	57.9

6.3 Absolute Final HPWL at Best Cooling Rate

Table 3 gives the absolute final HPWL at $\alpha = 0.95$.

Greedy lookahead achieves the lowest final HPWL on four of the five designs. Baseline SA achieves the lowest on `design_3_large`.

6.4 Runtime

Table 4 reports mean runtime in seconds averaged over all five cooling rates. Baseline SA shows an unusual spike on `design_3_large` (12.10 s mean) because 500-cell designs fall just

Table 3: Final HPWL at $\alpha = 0.95$

Variant	Small	Medium	Large	Dense	Extreme
Baseline SA	2,169	3,397	6,092	12,307	21,352
Adaptive reheating	2,226	3,601	6,701	12,501	21,188
Force-directed init	2,183	3,662	6,647	12,522	21,421
Greedy lookahead	2,148	3,373	6,132	11,244	18,919
Range limiting	2,160	3,578	6,553	12,165	20,897

below the scaling threshold, using the full $20 \times N_{\text{cells}}$ move budget. Designs above 500 cells trigger the scaled-down budget and finish faster. Adaptive reheating and force-directed are the fastest overall. Greedy lookahead is the slowest, particularly on the three designs below 500 cells where the $k = 8$ lookahead budget applies.

Table 4: Mean runtime (s) averaged over all cooling rates

Variant	Small	Medium	Large	Dense	Extreme
Baseline SA	2.66	6.33	12.10	1.59	1.57
Adaptive reheating	1.53	1.61	1.79	2.10	2.01
Force-directed init	1.14	1.26	1.39	1.49	1.51
Greedy lookahead	8.22	8.92	10.06	5.40	5.38
Range limiting	4.34	4.53	4.59	4.45	4.19

6.5 Graphs

In Figure 1, the small and medium designs produce nearly flat curves where the difference in final HPWL between $\alpha = 0.75$ and $\alpha = 0.95$ is only around 100–330 units, meaning the cooling rate has little practical effect on small designs. In the dense and extreme designs, the final HPWL drops by roughly 3,350 and 5,500 units respectively across the same cooling rate range, showing that larger designs benefit significantly from a slower schedule. This is because high cell counts give the annealer more ways to escape local optima, and those escapes only happen when the temperature is kept high long enough.

Figure 2 shows how total wire length changes with temperature during the annealing schedule at $\alpha = 0.95$. All five designs follow the same trend where HPWL drops rapidly during the high-temperature phase as large improving moves are accepted freely, then levels off as temperature falls and acceptance of worsening moves becomes increasingly rare. The steepest decrease occurs in the middle of the temperature range, where the acceptance probability transitions from near 1.0 to near 0.0. The larger designs start and end at higher absolute HPWL values but their relative improvement over the schedule is comparable to the smaller designs.

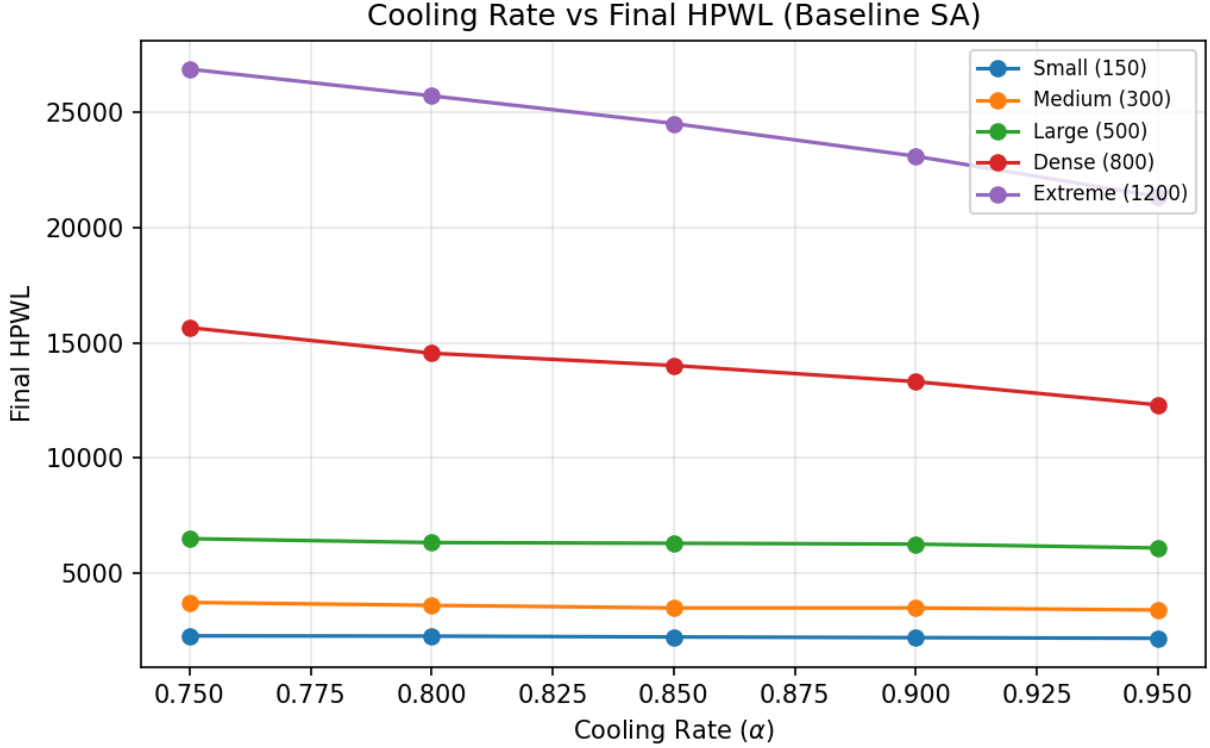


Figure 1: Cooling rate vs final HPWL for the baseline SA variant across all five benchmark designs.

7 Discussion

7.1 Cooling Rate Sensitivity

Baseline SA improves from 53.1% at $\alpha = 0.75$ to 62.7% at $\alpha = 0.95$ on the extreme design, a gap of nearly 10 percentage points. On the small design the same gap is only 1.6 points, reflecting the smaller search space. Cooling rates below 0.85 frequently terminate via early stopping before the schedule reaches T_{final} , which explains the steeper quality drop at $\alpha = 0.75$ on larger designs.

7.2 Variant Comparison

Greedy lookahead achieves the lowest absolute final HPWL on four of five designs and consistently outperforms the baseline by 1–3 percentage points in improvement. The gains are most significant on the extreme design, where the final HPWL of 18,919 is 11.4% lower than the baseline’s 21,352. The cost is runtime: greedy lookahead is 3–5 $\times$ slower than the baseline on designs below the 500-cell scaling threshold. On the dense and extreme designs the lookahead budget drops to $k = 4$, which cuts the runtime penalty roughly in half.

Adaptive reheating is the fastest variant across all designs. Despite its low runtime, it is competitive with or slightly below the baseline in absolute final HPWL on the extreme

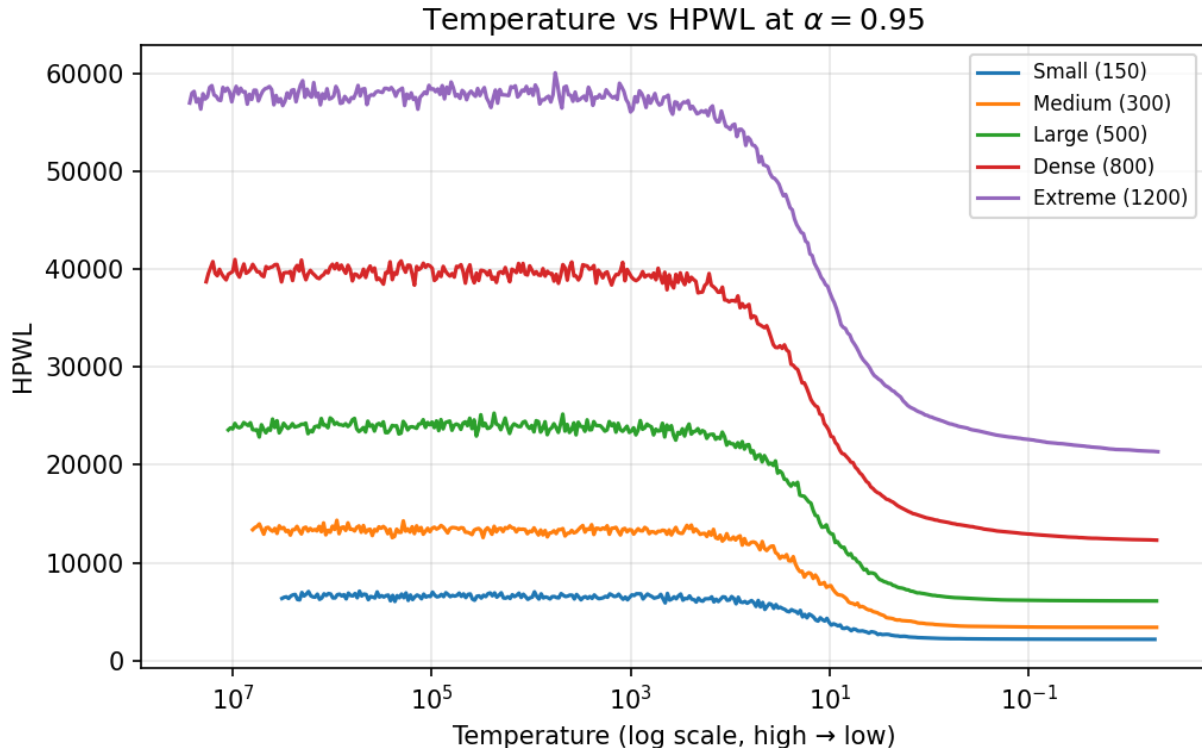


Figure 2: Temperature vs HPWL at $\alpha = 0.95$ for all five benchmark designs. Temperature decreases left to right on the log scale.

and dense designs (21,188 vs. 21,352 on extreme at $\alpha = 0.95$). The reheating mechanism compensates for the aggressive early cooling by periodically raising temperature when the acceptance rate falls below 10%, allowing the search to escape stagnation without running a full slow schedule.

Force-directed initialization produces a starting HPWL that is 26–34% lower than the random starting point used by the other variants. Despite this head start, its final HPWL at $\alpha = 0.95$ is comparable to but slightly worse than the baseline on most designs. This suggests that force-directed placement organizes cells near globally good positions but the SA refinement gains from a low-quality random starting point are larger in absolute terms, since there is more room to improve.

Range limiting achieves the most consistent runtime across all design sizes (roughly 4–5 seconds regardless of cell count), which is its primary advantage. Quality is slightly below the baseline on most designs, but the runtime predictability makes it a practical choice when wall-clock time is a hard constraint.

7.3 Implementation Notes

Type legality is maintained by construction throughout the run. Every move targets a site or cell of the matching type, so there is no legality check overhead in the inner loop.

The temperature schedule normalizes T_{final} by N_{nets} so the stopping criterion scales consistently across designs of different sizes. The runtime anomaly on **design_3_large** (Table 4) is a direct consequence of the $N_{\text{cells}} = 500$ scaling boundary: the moves-per-temperature budget is capped at $20 \times N_{\text{cells}}$ below this threshold, causing the large design to run the most moves per step despite not being the largest design.